МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 1 з дисципліни "Паралельні та розподілені обчислення" на тему „ Підготовка до виконання проєктної роботи з теми OpenMP №2"

Виконав

студент 2 курсу

групи ФЕП-23

Чепара Станіслав Богданович

Перевірив

ассистент кафедри СП

Кулик Петро Русланович

Львів – 2026

Індивідуальне завдання

Варіант 2: написати паралельну OpenMP програму, яка знаходить суму елементів матриці.

Що було зроблено

У ході роботи було написано програму **matrix_sum_openmp.c**, яка:

- створює квадратну матрицю;
- заповнює матрицю псевдовипадковими числами;
- обчислює суму елементів матриці послідовно;
- обчислює суму елементів матриці паралельно за допомогою OpenMP;
- використовує директиву **#pragma omp parallel for** і опцію **reduction(+:sum)**;
- вимірює час виконання за допомогою **omp_get_wtime()**;
- показує, який потік яку ітерацію виконує для **schedule(static)**, **schedule(dynamic, 1)** і **schedule(guided, 2)**.

Результат

Програму було успішно скомпільовано і перевірено.

Для матриць розміру **1000x1000**, **2000x2000** і **4000x4000** було виконано послідовний і паралельний підрахунок суми. У всіх випадках результати збіглися, тобто перевірка дала **OK**.

Було також перевірено роботу **schedule(static)**, **schedule(dynamic, 1)** і **schedule(guided, 2)**. Під час виконання програма виводить повідомлення у форматі:

```
[номер потоку]: calculation of the iteration number номер_ітерації.
```

```
Finding the sum of matrix elements

Matrix size: 1000 x 1000
Sequential sum = 49498583, time = 0.000977 s
Threads = 1, parallel sum = 49498583, time = 0.000959 s, speedup = 1.019079, check = OK
Threads = 2, parallel sum = 49498583, time = 0.000634 s, speedup = 1.540960, check = OK
Threads = 4, parallel sum = 49498583, time = 0.000454 s, speedup = 2.151753, check = OK
Threads = 8, parallel sum = 49498583, time = 0.000400 s, speedup = 2.441132, check = OK

Matrix size: 2000 x 2000
Sequential sum = 198022445, time = 0.003287 s
Threads = 1, parallel sum = 198022445, time = 0.003540 s, speedup = 0.928618, check = OK
Threads = 2, parallel sum = 198022445, time = 0.002106 s, speedup = 1.560991, check = OK
Threads = 4, parallel sum = 198022445, time = 0.001381 s, speedup = 2.381088, check = OK
Threads = 8, parallel sum = 198022445, time = 0.001073 s, speedup = 3.064854, check = OK

Matrix size: 4000 x 4000
Sequential sum = 792128424, time = 0.014180 s
Threads = 1, parallel sum = 792128424, time = 0.014564 s, speedup = 0.973681, check = OK
Threads = 2, parallel sum = 792128424, time = 0.008221 s, speedup = 1.724778, check = OK
Threads = 4, parallel sum = 792128424, time = 0.005487 s, speedup = 2.584281, check = OK
Threads = 8, parallel sum = 792128424, time = 0.003794 s, speedup = 3.737534, check = OK
```

```
Distribution of iterations between threads:

Schedule(static)
[0]: calculation of the iteration number 0.
[0]: calculation of the iteration number 1.
[2]: calculation of the iteration number 4.
[2]: calculation of the iteration number 5.
[1]: calculation of the iteration number 2.
[1]: calculation of the iteration number 3.
[3]: calculation of the iteration number 6.
[3]: calculation of the iteration number 7.

Schedule(dynamic, 1)
[3]: calculation of the iteration number 1.
[2]: calculation of the iteration number 0.
[3]: calculation of the iteration number 4.
[2]: calculation of the iteration number 5.
[2]: calculation of the iteration number 7.
[3]: calculation of the iteration number 6.
[0]: calculation of the iteration number 2.
[1]: calculation of the iteration number 3.

Schedule(guided, 2)
[0]: calculation of the iteration number 0.
[0]: calculation of the iteration number 1.
[1]: calculation of the iteration number 2.
[1]: calculation of the iteration number 3.
[3]: calculation of the iteration number 6.
[3]: calculation of the iteration number 7.
[2]: calculation of the iteration number 4.
[2]: calculation of the iteration number 5.
```

Висновок

У лабораторній роботі було реалізовано паралельне знаходження суми елементів матриці засобами OpenMP. Для правильного підсумовування використано **reduction**. Час виконання було виміряно для різної кількості потоків і різних розмірів матриці. Отримані результати показали правильну роботу програми і коректний розподіл ітерацій між потоками.